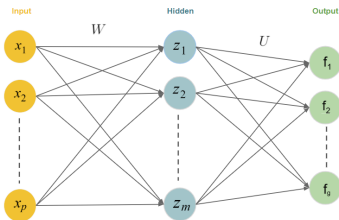


Deep Learning

Single Hidden Layer Networks for Multi-Class Classification



Learning goals

- Neural network architectures for multi-class classification
- Softmax activation function
- Softmax loss

MULTI-CLASS CLASSIFICATION

- We have only considered regression and binary classification problems so far.
- How can we get a neural network to perform multiclass classification?

MULTI-CLASS CLASSIFICATION

- The first step is to add additional neurons to the output layer.
- Each neuron in the layer will represent a specific class (number of neurons in the output layer = number of classes).

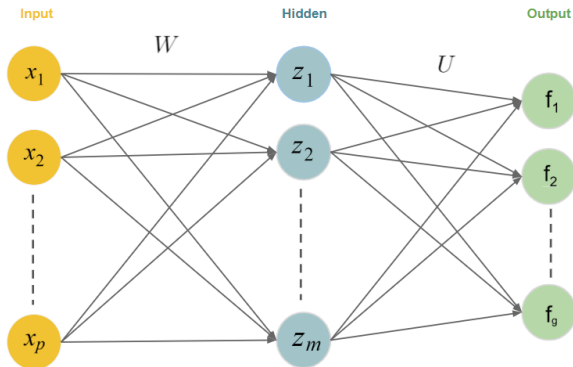


Figure: Structure of a single hidden layer, feed-forward neural network for g -class classification problems (bias term omitted).

MULTI-CLASS CLASSIFICATION

Notation:

- For g -class classification, g output units:

$$\mathbf{f} = (f_1, \dots, f_g)$$

- m hidden neurons $z_1, \dots, z_m$, with

$$z_j = \sigma(\mathbf{W}_j^T \mathbf{x}), \quad j = 1, \dots, m.$$

- Compute linear combinations of derived features z :

$$f_{in,k} = \mathbf{U}_k^T \mathbf{z}, \quad \mathbf{z} = (z_1, \dots, z_m)^T, \quad k = 1, \dots, g$$

MULTI-CLASS CLASSIFICATION

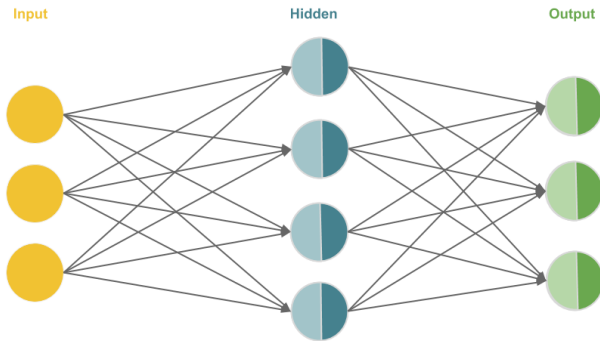
- The second step is to apply a **softmax** activation function to the output layer.
- This gives us a probability distribution over g different possible classes:

$$f_{out,k} = \tau_k(f_{in,k}) = \frac{\exp(f_{in,k})}{\sum_{k'=1}^g \exp(f_{in,k'})}$$

- This is the same transformation used in softmax regression!
- Derivative $\frac{\partial \tau(\mathbf{f}_{in})}{\partial \mathbf{f}_{in}} = \text{diag}(\tau(\mathbf{f}_{in})) - \tau(\mathbf{f}_{in})\tau(\mathbf{f}_{in})^T$
- It is a “smooth” approximation of the argmax operation, so $\tau((1, 1000, 2)^T) \approx (0, 1, 0)^T$ (picks out 2nd element!).

MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



$$\begin{pmatrix} 3 & -9 & 2 \\ 11 & -2 & 7 \\ -6 & 3 & -4 \\ 6 & -1 & 5 \end{pmatrix} \begin{pmatrix} 5 \\ 2 \\ -1 \\ 1 \end{pmatrix}$$

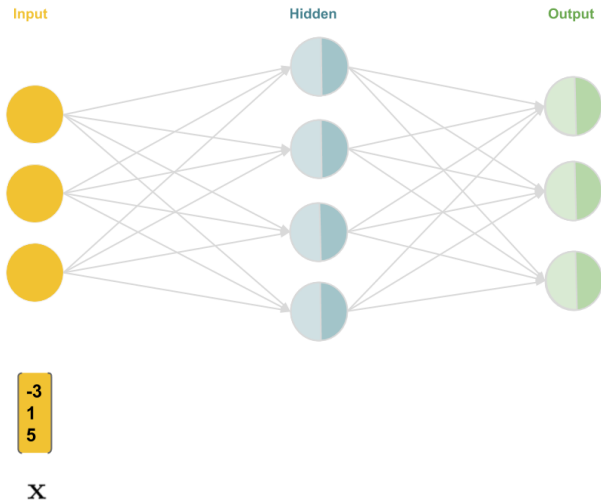
W^T b

$$\begin{pmatrix} 3 & -12 & 8 & 1 \\ 2 & -3 & 9 & 1 \\ -5 & 1 & -1 & 7 \end{pmatrix} \begin{pmatrix} 6 \\ 0 \\ -8 \end{pmatrix}$$

U^T c

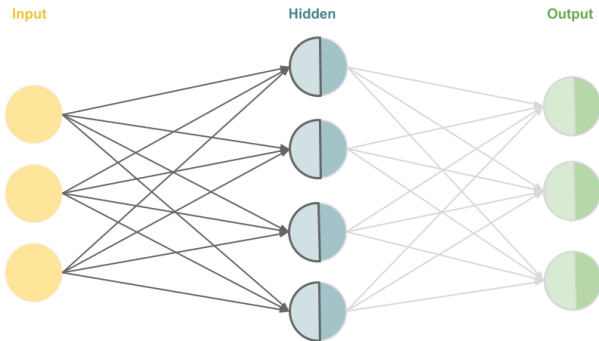
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



MULTI-CLASS CLASSIFICATION: EXAMPLE

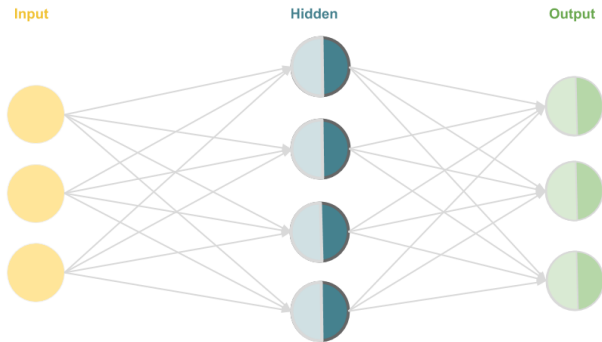
Forward pass (Hidden: Sigmoid, Output: Softmax).



$$\begin{bmatrix} -3 \\ 1 \\ 5 \end{bmatrix} \quad \begin{pmatrix} (-3)*3 + 1*(-9) + 5*2 + 5 \\ (-3)*11 + 1*(-2) + 5*7 + 2 \\ (-3)*(-6) + 1*3 + 5*(-4) + (-1) \\ (-3)*6 + 1*(-1) + 5*5 + 1 \end{pmatrix}$$
$$\mathbf{x} \quad \mathbf{z}_{in} = \mathbf{w}^T \mathbf{x} + \mathbf{b}$$

MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



$\begin{bmatrix} -3 \\ 1 \\ 5 \end{bmatrix}$

$\mathbf{x}$

$\begin{bmatrix} -3 \\ 2 \\ 0 \\ 7 \end{bmatrix}$

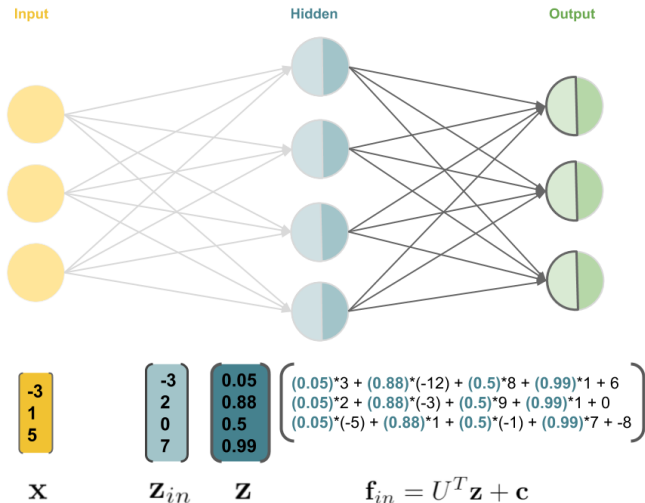
$\mathbf{z}_{in}$

$\begin{pmatrix} 1 / (1 + \exp(-3)) \\ 1 / (1 + \exp(-2)) \\ 1 / (1 + \exp(0)) \\ 1 / (1 + \exp(-7)) \end{pmatrix}$

$\mathbf{z} = \mathbf{z}_{out} = \sigma(\mathbf{z}_{in})$

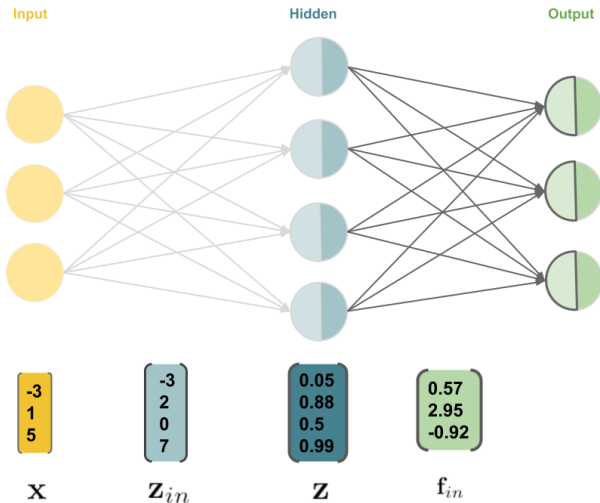
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



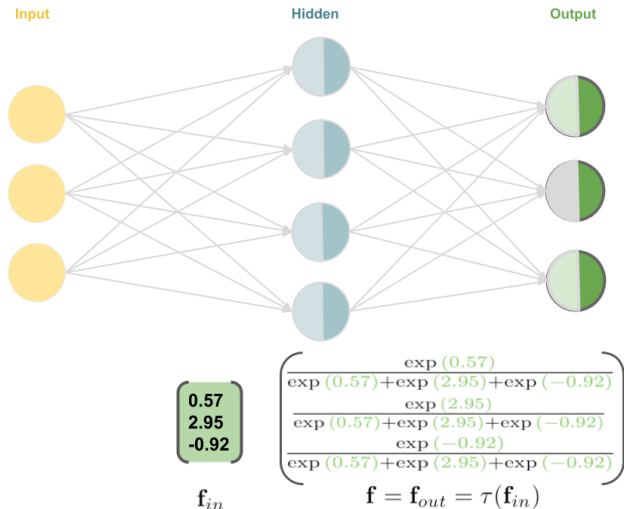
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



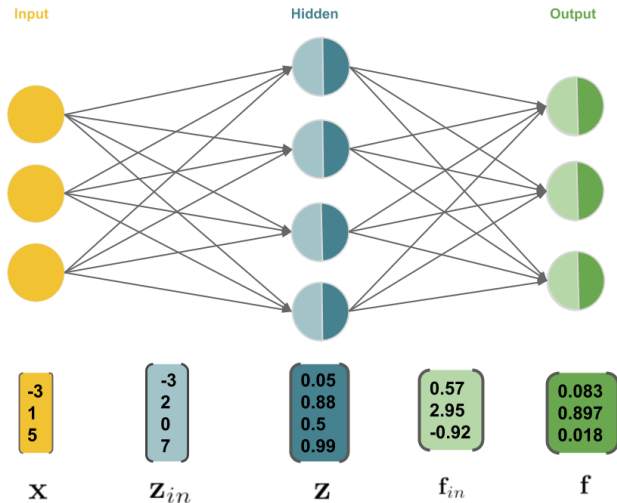
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



OPTIMIZATION: SOFTMAX LOSS

- The loss function for a softmax classifier is

$$L(y, f(\mathbf{x})) = - \sum_{k=1}^g [y = k] \log \left(\frac{\exp(f_{in,k})}{\sum_{k'=1}^g \exp(f_{in,k'})} \right)$$

$$\text{where } [y = k] = \begin{cases} 1 & \text{if } y = k \\ 0 & \text{otherwise} \end{cases}.$$

- This is equivalent to the cross-entropy loss when the label vector $\mathbf{y}$ is one-hot coded (e.g. $\mathbf{y} = (0, 0, 1, 0)^T$).
- Optimization: Again, there is no analytic solution.